INSTITUTO POLITÉCNICO NACIONAL
ESCUELA SUPERIOR DE CÓMPUTO

Práctica 2: Exploring a Text

NATURAL LANGUAGE PROCESSING

7CM2

INTEGRANTES:

VÁZQUEZ BLANCAS CESAR SAID

PROFESOR:

JORGE LUIS ROSAS TRIGUEROS

FECHA DE ENTREGA: 17 DE MARZO DE 2026

FECHA DE REALIZACION DE LA PRACTICA: 8 DE MARZO DE 2026

Marco Teórico

El Procesamiento de Lenguaje Natural (Natural Language Processing, NLP) es una rama de la inteligencia artificial que se encarga de estudiar y desarrollar métodos que permitan a las computadoras comprender, interpretar y generar lenguaje humano de forma automática. Este campo combina conocimientos de informática, lingüística, estadística y aprendizaje automático para analizar grandes volúmenes de texto y extraer información útil (Jurafsky & Martin, 2023).

Uno de los problemas fundamentales dentro del procesamiento de texto es la representación y análisis estadístico de las palabras que aparecen en un documento. Para realizar este análisis es necesario transformar el texto en una estructura que pueda ser procesada por algoritmos computacionales. Este proceso se conoce como preprocesamiento de texto y constituye una etapa esencial en la mayoría de los sistemas de análisis de lenguaje natural (Bird, Klein & Loper, 2009).

El preprocesamiento de texto generalmente incluye varias etapas importantes. Una de las primeras es la normalización del texto, que consiste en convertir todas las palabras a minúsculas y eliminar caracteres innecesarios. Esto permite evitar que una misma palabra sea considerada como diferentes términos debido a variaciones en su escritura.

Otra etapa fundamental es la tokenización, que consiste en dividir el texto en unidades más pequeñas llamadas *tokens*. En la mayoría de los casos, estos tokens corresponden a palabras individuales. La tokenización permite que el texto pueda ser analizado por algoritmos que operan sobre secuencias de palabras o símbolos (Manning, Raghavan & Schütze, 2008).

Posteriormente se aplica la eliminación de palabras vacías o stopwords. Las stopwords son palabras muy frecuentes en un idioma que generalmente no aportan información relevante al significado general del texto. Ejemplos de estas palabras incluyen artículos, preposiciones y conjunciones como "the", "and", "of" en inglés o "el", "la", "de", "y" en español. La eliminación de estas palabras permite que el análisis se enfoque en términos con mayor contenido semántico (Jurafsky & Martin, 2023).

Una vez que el texto ha sido preprocesado, es posible realizar un análisis de frecuencia de palabras, el cual consiste en contar cuántas veces aparece cada término dentro de un documento o conjunto de documentos. Este tipo de análisis es una técnica básica pero muy importante en minería de texto y recuperación de información, ya que permite identificar los términos más relevantes dentro de un corpus (Manning et al., 2008).

En programación, el análisis de frecuencia de palabras suele implementarse mediante estructuras de datos como diccionarios o contadores, que permiten almacenar cada palabra junto con el número de veces que aparece. En Python, por ejemplo, la clase

Counter del módulo collections facilita este proceso al permitir contar automáticamente la frecuencia de elementos dentro de una lista.

Además, existen bibliotecas especializadas para trabajar con texto, como NLTK (Natural Language Toolkit), una de las plataformas más utilizadas para el aprendizaje y desarrollo de aplicaciones de procesamiento de lenguaje natural en Python. NLTK incluye herramientas para tokenización, análisis sintáctico, manejo de stopwords y cálculo de distribuciones de frecuencia, lo que la convierte en una herramienta fundamental para la experimentación y desarrollo de algoritmos de análisis lingüístico (Bird et al., 2009).

En la práctica desarrollada en este laboratorio se aplican varios de estos conceptos fundamentales del procesamiento de lenguaje natural. A partir de un archivo de texto seleccionado, se realiza un proceso de preprocesamiento que incluye normalización del texto, tokenización mediante expresiones regulares, eliminación de stopwords en inglés y español, y finalmente el cálculo de la frecuencia de las palabras. El objetivo es identificar las 100 palabras más frecuentes del texto, lo cual permite obtener una representación estadística del contenido del documento analizado.

Este tipo de técnicas constituye la base de múltiples aplicaciones modernas de inteligencia artificial, como motores de búsqueda, análisis de sentimientos, clasificación de documentos, sistemas de recomendación y modelos de lenguaje utilizados en asistentes virtuales y sistemas de conversación.

Material y Equipo

Para la realización de esta práctica se utilizaron diferentes herramientas de software y hardware que permitieron llevar a cabo el análisis de texto y el cálculo de la frecuencia de palabras dentro de un documento.

Hardware

Computadora personal

Se utilizó una computadora personal con capacidad suficiente para ejecutar programas en Python y procesar archivos de texto. El análisis realizado en esta práctica no requiere recursos computacionales muy elevados, ya que el procesamiento consiste principalmente en operaciones de lectura de archivos, manipulación de cadenas de texto y conteo de frecuencias.

Software

Python 3

Python es un lenguaje de programación de alto nivel ampliamente utilizado en el área de ciencia de datos, inteligencia artificial y procesamiento de lenguaje natural. Su sintaxis clara y la gran cantidad de bibliotecas disponibles lo convierten en una herramienta ideal para el análisis de texto y el desarrollo de algoritmos experimentales.

Editor de código

Para escribir y ejecutar el programa se utilizó un editor de código o entorno de desarrollo integrado (IDE), como Visual Studio Code, IDLE o cualquier otro editor compatible con Python. Estas herramientas facilitan la escritura, depuración y ejecución del código.

NLTK (Natural Language Toolkit)

NLTK es una de las bibliotecas más utilizadas para el procesamiento de lenguaje natural en Python. Proporciona herramientas para realizar tareas como tokenización, análisis sintáctico, eliminación de stopwords, análisis de frecuencia de palabras y manipulación de corpus lingüísticos. En esta práctica se utilizó NLTK para obtener listas de palabras vacías y para calcular distribuciones de frecuencia de palabras.

Archivo de texto (text2.txt)

El análisis se realizó sobre un archivo de texto que contiene el contenido que será procesado por el programa. Este archivo puede provenir de diversas fuentes, como libros digitales, artículos o textos descargados de repositorios públicos como Project Gutenberg. El archivo sirve como corpus de entrada para realizar el análisis de frecuencia de palabras.

Librerías utilizadas en el programa

Durante el desarrollo del script se utilizaron diversas librerías de Python que permiten manipular texto y analizar datos.

re (Expresiones regulares)
Esta biblioteca permite buscar patrones dentro de cadenas de texto. En esta práctica se utilizó para extraer únicamente las palabras del texto, ignorando signos de puntuación y caracteres especiales.

nltk.corpus.stopwords
Este módulo proporciona listas de palabras vacías en diferentes idiomas. Estas palabras se eliminan del análisis para concentrarse únicamente en los términos que aportan mayor significado.

collections.Counter
Counter es una estructura de datos especializada que permite contar automáticamente cuántas veces aparece cada elemento dentro de una colección. En esta práctica se utilizó para calcular la frecuencia de aparición de cada palabra dentro del texto.

nltk.probability.FreqDist
FreqDist es una herramienta incluida en NLTK que permite generar distribuciones de frecuencia de palabras. Esta estructura facilita el análisis estadístico del texto y permite identificar las palabras más comunes dentro del corpus analizado.

Desarrollo de la Practica

La práctica consistió en desarrollar un programa en Python capaz de analizar un archivo de texto y mostrar las 100 palabras más frecuentes dentro del documento. Para lograrlo, se aplicaron diversas técnicas de procesamiento de texto utilizando bibliotecas especializadas.

1. Importación de librerías

El primer paso fue importar las librerías necesarias para el procesamiento del texto.

```
import re
import nltk
from collections import Counter
from nltk.corpus import stopwords
```

Figura 1: Librerías a utilizar

Cada una de estas librerías cumple una función específica dentro del programa. La biblioteca re se utiliza para trabajar con expresiones regulares, las cuales permiten identificar patrones dentro del texto y extraer únicamente las palabras.

La biblioteca nltk proporciona herramientas especializadas para el procesamiento de lenguaje natural, incluyendo listas de palabras vacías (*stopwords*).

El módulo Counter del paquete *collections* permite contar la frecuencia de aparición de elementos dentro de una lista de manera eficiente.

2. Descarga del conjunto de stopwords

Posteriormente se descargó el conjunto de palabras vacías proporcionado por NLTK.

```
nltk.download('stopwords')
```

Figura 2: Descarga de las stopwords

Las *stopwords* son palabras muy comunes dentro de un idioma que generalmente no aportan información relevante para el análisis del texto. Al eliminarlas, el análisis se enfoca en palabras con mayor significado semántico.

3. Definición del conjunto de stopwords

En el programa se utilizaron listas de palabras vacías tanto en inglés como en español.

```
stop_words = set(stopwords.words('english')) | set(stopwords.words('spanish'))
```

Figura 3: Definición de stopwords en español e ingles

Esto permite que el programa elimine palabras comunes de ambos idiomas, lo cual es útil en textos que contienen contenido bilingüe o palabras mezcladas de diferentes idiomas.

4. Lectura del archivo de texto

Posteriormente se abrió el archivo de texto que contiene el contenido a analizar.

```
with open("text2.txt", "r", encoding="utf-8") as file:
    text = file.read().lower()
```

Figura 4: Lectura de archivo

El archivo se lee completamente y su contenido se convierte a minúsculas mediante el método lower(). Esto evita que una misma palabra sea considerada diferente únicamente por diferencias entre mayúsculas y minúsculas.

5. Extracción de palabras mediante expresiones regulares

El siguiente paso consiste en extraer las palabras del texto utilizando una expresión regular.

```
words = re.findall(r'\b[a-zA-ZáéíóúÁÉÍÓÚñÑüÜ]+\b', text)
```

Figura 5: Regex para extraer palabras

Esta expresión regular identifica secuencias de letras que corresponden a palabras válidas, incluyendo caracteres propios del español como acentos y la letra ñ. De esta manera se eliminan automáticamente signos de puntuación, números y otros caracteres que no forman parte de las palabras.

6. Eliminación de stopwords

Después se eliminan las palabras vacías utilizando una comprensión de listas.

```
filtered_words = [word for word in words if word not in stop_words]
```

Figura 6: Filtrado de palabras para quitar stopwords

Este proceso recorre todas las palabras extraídas del texto y conserva únicamente aquellas que no pertenecen al conjunto de *stopwords*. El resultado es una lista de palabras más relevantes para el análisis.

7. Cálculo de la frecuencia de palabras

Una vez obtenida la lista de palabras filtradas, se procede a contar cuántas veces aparece cada una.

```
word_count = Counter(filtered_words)

top_words = word_count.most_common(100)
```

Figura 7: Calculo de frecuencia y top 100

La clase Counter genera automáticamente un diccionario donde cada palabra se asocia con el número de veces que aparece en el texto. Posteriormente se utiliza el método most_common(100) para obtener únicamente las cien palabras más frecuentes.

8. Visualización de resultados

Finalmente se imprimen las palabras más frecuentes junto con su número de apariciones.

```
for word, count in top_words:
    print(word, count)
```

Figura 8: Recorrido para mostrar el top 100

Este ciclo recorre la lista de palabras más frecuentes y muestra cada palabra acompañada de su frecuencia dentro del texto analizado.

9. Distribución de frecuencia con NLTK

Adicionalmente, se utilizó la clase FreqDist de NLTK para generar una distribución de frecuencia de las palabras.

Posteriormente se muestran nuevamente las 100 palabras más frecuentes.

```
from nltk.probability import FreqDist

frequency_words = FreqDist(filtered_words)

print("\nLas 100 palabras más frecuentes (excluyendo stopwords):")
for word, frequency in frequency_words.most_common(100):
    print(f"{word}: {frequency}")
```

Figura 9: Mismo proceso, pero con uso de FreqDist

Esta herramienta permite realizar análisis estadísticos adicionales del texto y facilita la visualización de la distribución de palabras dentro del documento.

Resultados obtenidos

Al ejecutar el programa, el sistema procesa el archivo de texto seleccionado y genera una lista con las 100 palabras más frecuentes, excluyendo las palabras vacías. Estos resultados permiten identificar los términos que aparecen con mayor frecuencia en el documento analizado, lo cual proporciona una visión general del contenido del texto.

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\vbcs1\AppData\Roaming\nltk_data..
[nltk_data]   Package stopwords is already up-to-date!
quijote 2245
sancho 2174
si 1968
dijo 1808
tan 1245
señor 1065
así 1065
respondió 1063
ser 1059
bien 1051
merced 900
pues 865
sino 694
dos 685
caballero 677
decir 578
hacer 535
dios 531
aunque 531
señora 518
aquí 516
aquel 487
mal 463
cosa 447
buen 446
verdad 436
tal 428
allí 421
```

Figura 10: Primeros resultados del libro" Don Quijote"

```
Las 100 palabras más frecuentes (excluyendo stopwords):
quijote: 2245
sancho: 2174
si: 1968
dijo: 1808
tan: 1245
señor: 1065
así: 1065
respondió: 1063
ser: 1059
bien: 1051
merced: 900
pues: 865
sino: 694
dos: 685
caballero: 677
decir: 578
hacer: 535
dios: 531
aunque: 531
señora: 518
aquí: 516
aquel: 487
mal: 463
cosa: 447
buen: 446
verdad: 436
tal: 428
allí: 421
ver: 410
mundo: 396
```

Figura 11: Primeros resultados del libro" Don Quijote" con el uso de FreqDist

```
[nltk_data] Downloading package stopwords to
[nltk_data]     C:\Users\vbcs1\AppData\Roaming\nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
mr 807
elizabeth 645
could 531
would 482
darcy 430
said 406
mrs 353
bennet 339
much 336
must 322
miss 315
bingley 310
jane 303
one 292
know 242
though 238
well 230
never 228
think 222
may 218
soon 216
sister 216
good 208
might 206
time 201
wickham 200
lady 198
little 190
```

Figura 10: Primeros resultados del libro" Pride and Prejudice"

```
Las 100 palabras más frecuentes (excluyendo stopwords):
mr: 807
elizabeth: 645
could: 531
would: 482
darcy: 430
said: 406
mrs: 353
bennet: 339
much: 336
must: 322
miss: 315
bingley: 310
jane: 303
one: 292
know: 242
though: 238
well: 230
never: 228
think: 222
may: 218
soon: 216
sister: 216
good: 208
might: 206
time: 201
wickham: 200
lady: 198
little: 190
collins: 190
every: 185
```

Figura 11: Primeros resultados del libro" Pride and Prejudice" con el uso de FreqDist

Conclusiones y recomendaciones

Durante el desarrollo de esta práctica fue posible aplicar conceptos fundamentales del procesamiento de lenguaje natural mediante el uso del lenguaje de programación Python y la biblioteca NLTK. A través del análisis de un archivo de texto se logró identificar las palabras más frecuentes dentro del documento, utilizando técnicas de preprocesamiento como la conversión del texto a minúsculas, la tokenización mediante expresiones regulares y la eliminación de palabras vacías o *stopwords*.

El uso de herramientas como Counter y FreqDist permitió calcular de manera eficiente la frecuencia de aparición de cada palabra, lo que facilita la obtención de información estadística acerca del contenido del texto. Este tipo de análisis resulta útil para comprender de forma general los temas principales presentes en un documento, así como para identificar términos relevantes dentro de un corpus textual.

Además, esta práctica permitió comprender la importancia del preprocesamiento de datos en el análisis de texto. Sin este proceso, los resultados obtenidos podrían incluir palabras irrelevantes o duplicadas debido a diferencias en mayúsculas, puntuación o palabras comunes que no aportan significado al análisis.

Como recomendación, se podrían implementar mejoras al programa desarrollado, como la generación de gráficas de frecuencia de palabras, la creación de nubes de palabras (word clouds) o el análisis de textos más extensos para obtener resultados más representativos. También sería posible aplicar técnicas adicionales de procesamiento de lenguaje natural como lemmatización, stemming o análisis de sentimiento, lo cual permitiría realizar estudios más profundos sobre el contenido de los textos.

Finalmente, esta práctica demuestra cómo herramientas relativamente simples de programación pueden ser utilizadas para realizar análisis básicos de lenguaje natural, los cuales constituyen la base de sistemas más avanzados utilizados actualmente en áreas como minería de texto, motores de búsqueda, inteligencia artificial y análisis de datos.

Además de los ejercicios propuestos en la práctica, se realizaron experimentos adicionales para evaluar el comportamiento del programa con diferentes tipos de texto. En particular, se probó el script con textos tanto en inglés como en español, lo cual permitió verificar que la eliminación de *stopwords* funciona correctamente en ambos idiomas gracias al uso combinado de las listas de palabras vacías proporcionadas por la biblioteca NLTK. Esto permitió obtener resultados más representativos al eliminar palabras comunes como artículos, preposiciones y conjunciones en ambos idiomas.

También se realizó una comparación utilizando dos métodos distintos para calcular la frecuencia de palabras: el uso de la clase Counter de la biblioteca collections y el uso de la estructura FreqDist de la biblioteca NLTK. Ambos métodos permitieron obtener las 100 palabras más frecuentes del texto, lo que permitió confirmar que los resultados generados por ambos enfoques eran consistentes. Esta experimentación ayudó a comprender que existen diferentes herramientas dentro de Python para resolver el mismo problema, y que la elección de una u otra depende del contexto del análisis y de las funcionalidades adicionales que se requieran.

A partir de estas pruebas adicionales se pudo observar la importancia del preprocesamiento del texto y la utilidad de las bibliotecas especializadas en procesamiento de lenguaje natural para realizar análisis estadísticos sobre documentos de manera eficiente.

Bibliografía

- Bird, S., Klein, E., & Loper, E. (2009).
  Natural Language Processing with Python.
  O'Reilly Media.
- Jurafsky, D., & Martin, J. H. (2023).
  Speech and Language Processing (3rd ed. draft).
  Stanford University.
- Manning, C. D., Raghavan, P., & Schütze, H. (2008).
  Introduction to Information Retrieval.
  Cambridge University Press.
- NLTK Project. (2024).
  Natural Language Toolkit Documentation.
  Disponible en: https://www.nltk.org/
- Project Gutenberg. (2024).
  Free eBooks – Digital Library.
  Disponible en: https://www.gutenberg.org/
- Python Software Foundation. (2024).
  Python Standard Library Documentation.
  Disponible en: https://docs.python.org/3/